NLQ Paper

Agentar-Scale #1:

1. **Task Understanding** (offline + online) → thu thập context (evidence + few-shot).
2. **SQL Generation Scaling** → sinh nhiều candidate SQL bằng 3 hình thức scaling:
   - **Internal Scaling**: Intrinsic Reasoning Generator (suy nghĩ sâu, chain-of-thought).
   - **Parallel Scaling**: Diverse Synthesis (sinh đa dạng).
   - **Sequential Scaling**: Iterative Refinement (sửa chữa lặp).
3. **SQL Selection Scaling** → chọn candidate tốt nhất bằng Tournament + Reasoning Selector.

Triết lý cốt lõi: Kết hợp "suy nghĩ sâu" (reasoning) + "sinh đa dạng" (diversity) + "tinh chỉnh" (refinement) + "so sánh" (selection) để model "nghĩ lâu hơn, thử nhiều hơn, chọn lọc tốt hơn" chỉ bằng computation tại inference.

2. Vector Store được xây dựng và trích xuất như thế nào?

Đây là phần **Task Understanding (3.1)** – giai đoạn preprocessing quan trọng nhất.

Xây dựng vector store (offline preprocessing):

- Paper tạo **2 vector store riêng biệt**:
  - **V_D_cell**: Chứa **tất cả các giá trị cell textual** trong database (tức là dữ liệu thực tế trong các bảng, không phải schema).
  - **V_D_example**: Chứa **toàn bộ training set** dưới dạng few-shot examples (câu hỏi + SQL tương ứng).
- **Embedding model**: Sử dụng **all-MiniLM-L6-v2** (từ Sentence Transformers) – một model nhẹ, nhanh, chuyên cho semantic search.
- **Quy trình indexing**:

- Mỗi cell value trong database → embed thành vector → lưu vào V_D_cell.
- Mỗi training example (question + SQL) → embed → lưu vào V_D_example.

3. Cách xây dựng luồng suy nghĩ chi tiết (Intrinsic Reasoning Generator)

Đây là **Internal Scaling** – phần "suy nghĩ sâu" nhất của paper.

Mô hình: Omni-SQL-32B được **fine-tune bằng RL (GRPO)** train bằng **tập train của BIRD**

Tư tưởng của tập train là cho mô hình gen ra nhiều câu SQL cho các câu hỏi và đánh điểm cho các câu theo công thức sau:

exploration and stability.

Reward Design. We define a reward function R_G solely on final execution correctness and basic syntax validity following Arctic-Text2SQL-R1:

$$R_G = \begin{cases} 1, & \text{if results match the ground truth;} \\ 0.1, & \text{if the SQL is executable;} \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Automatic Metadata for SQL:

1. Tổng quan luồng suy nghĩ (Overall Pipeline / Orchestrated Metadata Flow)

Paper **không dùng LLM để "nghĩ sâu" tại test-time** như Agentar-Scale-SQL, mà tập trung **pre-processing** (tự động hóa việc "hiểu database" trước khi sinh SQL).

Triết lý cốt lõi:
"Khó nhất trong Text-to-SQL thực tế **không phải viết SQL**, mà là **hiểu database**

(schema thiếu tài liệu, join ẩn, công thức kinh doanh, giá trị mặc định lạ...). Metadata do con người (oracle trong BIRD) quá đắt và không scalable. → Hãy tự động trích xuất metadata từ chính database + log + LLM, rồi fuse lại để LLM sinh SQL chính xác hơn."

Luồng chính (xem Figure 1 trong paper – pipeline tổng thể):

1. **Profiling** (offline): Truy vấn database để lấy statistical metadata + MinHash sketches.
2. **Query Log Analysis** (offline): Khai thác log SQL thực tế để tìm join ẩn, multi-field join, business logic.
3. **SQL-to-Text** (offline): Dùng LLM sinh câu hỏi tự nhiên từ SQL + metadata → tạo few-shot examples chất lượng cao.
4. **Schema Linking + Fusion** (online): Kết hợp 3 nguồn metadata → xây dựng "focused schema" (chỉ bảng/cột liên quan) → prompt LLM sinh SQL.
5. **Candidate Generation & Validation**: Sinh nhiều SQL candidate → validate bằng SQLglot + rule-based fix → chọn kết quả tốt nhất.

Kết quả thực tế: Đạt #1 BIRD leaderboard nhiều lần (9/2024, 11/2024, 3/2025) **không dùng oracle metadata**, vượt đối thủ 10.28 điểm phần trăm (67.41% vs 57.13% trên test set không oracle).

Ví dụ cụ thể từ pape

1. **Multi-field join (join nhiều cột)**
   Query Q782:
   colour.id = superhero.eye_colour_id AND colour.id = superhero.hair_colour_id
   → Phải join **2 cặp cùng lúc** mới đúng. Schema không ghi bất kỳ FK nào.
2.

5.4.2 Other Join Constraints

Many more join constraints can be found (examples from dev 6-24). For example, there are two multi-field joins:

Q782: colour.id=superhero.eye_colour_id And colour.id=superhero.hair_colour_id

Q1016: results.raceid=pitstops.raceid And results.driverid=pitstops.driverid

None of these 4 individual predicates are listed in the SQLite schema, although the hint in 782 suggests the join. We can also find joins that require computation: